

**KWAME NKRUMAH UNIVERSITY OF SCIENCE AND TECHNOLOGY,
KUMASI
COLLEGE OF SCIENCE
FACULTY OF PHYSICAL AND COMPUTATIONAL SCIENCES
DEPARTMENT OF MATHEMATICS**

**COURSE: SCIENTIFIC COMPUTING II
MID-SEMESTER PROJECT WORK
PROJECT REPORT, PART TWO**



**OBED ADU-GYAMFI
INDEX NUMBER: 8597421**

August, 2025

Contents

1	Introduction.	1
1.1	Overview.	1
1.2	Concepts.	2
2	Algorithms.	3
2.1	Mathematical Representation of Vector Multiplication, Scaler Product . . .	3
3	Test and Results	5
4	Performance Analysis	7
4.1	Efficiency	7
4.2	SpeedUp Analysis	7
5	conclusion	9
A	MPI Implementation the Dot product of two Vectors.	10

Chapter 1

Introduction.

1.1 Overview.

Scientists employ the use of very powerful and sophisticated systems to solve complex scientific and mathematical problems and computations that require large amounts of processing power, memory, and storage. These robust systems involve clusters, supercomputers, and cloud-based platforms. These systems are referred to as High Performance Computing, HPC. HPC systems integrate multiple computing resources to perform trillions of calculations per second, making them essential for scientific simulations, engineering design, financial modeling, weather prediction, data analytics, and artificial intelligence. One of the foundational concepts of HPC is parallel computing. Parallel computing is a paradigm in which a computational task is divided into smaller sub-tasks that are executed simultaneously across multiple processors or cores. Instead of performing one operation at a time (sequentially), parallel computing allows several operations to be carried out concurrently.

Parallelism is particularly effective for problems that can be decomposed into independent tasks, such as numerical simulations, image processing, and large-scale data analysis.

There are two main categories of parallel computing:

- Shared Memory Systems – multiple processors access the same memory space (e.g., multicore processors).
- Distributed Memory Systems – each processor has its own private memory, and processors communicate via a network (e.g., computer clusters).

In this project, we focus on the parallelization of vector-vector operations using MPI. A root process is designated to generate all the required data, partition it, and distribute appropriate segments to worker processes for computation, thereby eliminating duplication of data across ranks. The results are then gathered and combined to form the final output. The implementation was evaluated through scalability tests on large datasets, with particular emphasis on a problem size of $N = 2.5 \times 10^7$. Performance metrics such as execution

time, speedup, and efficiency were examined. The goal of this work is to demonstrate the performance benefits of parallel computing, assess the scalability of MPI-based vector operations, and provide insights into the efficiency of distributed memory programming in handling large-scale numerical computations.

1.2 Concepts.

Dot product, also known as the "scalar product", is a binary operation that takes two vectors and returns a scalar quantity. The dot product of two vectors can be defined as the product of the magnitudes of the two vectors and the cosine of the angle between the two vectors. Alternatively, it is defined as the product of the projection of the first vector onto the second vector and the magnitude of the second vector. In the context of this report, the algebraic definition of the dot product is employed.

The Message Passing Interface (MPI) is the most widely used standard for parallel programming on distributed memory systems. It provides a set of communication protocols and library routines that allow processes to exchange data and coordinate computation across different nodes in a cluster.

MPI supports both point-to-point communication (between two processes) and collective communication (involving groups of processes). It enables programmers to design parallel applications that scale efficiently from a few cores to thousands of nodes.

Speedup is calculated as the ratio of the serial execution time to the parallel execution time. This is represented mathematically as:

$$\text{Speedup}(s) = \frac{\text{Time by serial run (1 core)}}{\text{Time with n cores}}$$

It measures the effectiveness of parallelization. Ideally, speedup increases linearly with the number of cores, but practical factors such as communication overhead and memory contention affect scalability.

Chapter 2

Algorithms.

2.1 Mathematical Representation of Vector Multiplication, Scaler Product

Given two vectors $A = (a_1, a_2, a_3, \dots, a_n)$ and $B = (b_1, b_2, b_3, \dots, b_n)$, the dot product of the vectors A and B is defined algebraically as:

$$a \cdot b = \sum_{i=0}^n a_i b_i$$

Algorithm 1 Scalar Product of Two Vectors.

Input: Vectors $\mathbf{A} = (a_1, a_2, \dots, a_n)$, $\mathbf{B} = (b_1, b_2, \dots, b_n)$

Output: Scalar product $s = \mathbf{A} \cdot \mathbf{B}$

$s \leftarrow 0$ **for** $i \leftarrow 1$ **to** n **do**
 $s \leftarrow s + a_i \times b_i$

end

return s

```
1  #include "vector.h"
2
3  /**
4   * vector_vector - A vector-vector multiplication function
5   *
6   * Description: This function is a serial code that multiplies two
7   *               vectors and returns a scaler
8   *
9   * @m: This parameter is the dimension of the first vector A
```

```
9  * @n: This parameter also represents the dimension of the second
    vector B
10  * @vecA: This is a pointer to the first vector A
11  * @vecB: This is also a pointer to the second vector B.
12  *
13  * Return: A double.
14  */
15
16 double vector_vector(int m, int n, double *vecA, double *vecB){
17     double results = 0;
18     int i;
19     start = MPI_Wtime();
20
21     for (i = 0; i <= m; i++){
22         results += vecA[i] * vecB[i];
23     }
24     end = MPI_Wtime();
25     t_used = (double *)malloc(sizeof(double));
26     if (t_used != NULL){
27         *t_used = ((double)(end - start));
28     }
29     return (results);
30 }
```

Chapter 3

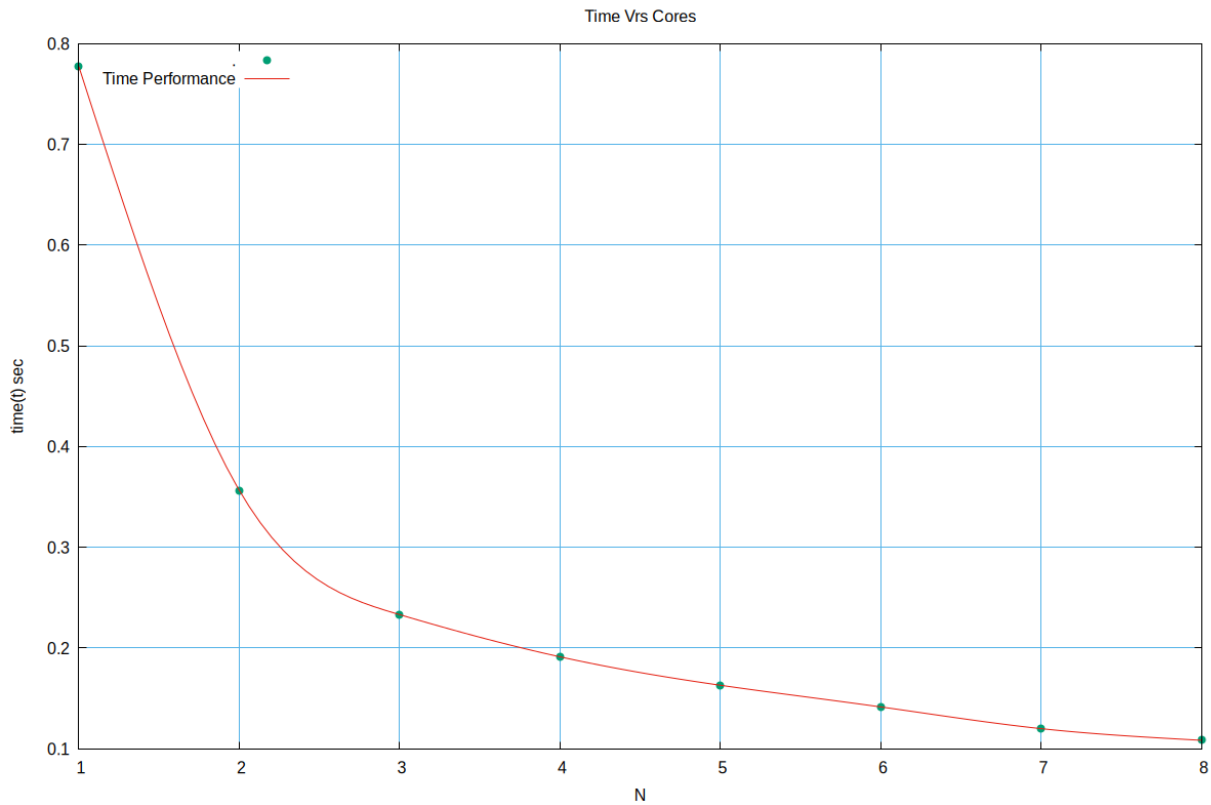
Test and Results

Below is the execution records of the time taken by n core(s) from a serial run, $n = 1$ core to $n = 8$ cores processors with a vector size (N), number of cores, and execution time. Scalability tests performed with $N = 2.5 \times 10^7$ for varying the number of cores.

Table 3.1: Execution Time of different Cores of Processors

N	Core	$V \cdot V/s$
2.5×10^7	1	0.777269
2.5×10^7	2	0.326578
2.5×10^7	3	0.233180
2.5×10^7	4	0.211311
2.5×10^7	5	0.167093
2.5×10^7	6	0.141583
2.5×10^7	7	0.120029
2.5×10^7	8	0.108472

Figure 3.1: Time Vrs Number of Cores



The results show how parallelization reduces computation time compared to the serial implementation. The implementation distributes data across multiple processes to leverage parallel computation and improve performance.

Chapter 4

Performance Analysis

Table 4.1: Performance Analysis Table

N	Core(s)	V·V/s	SpeedUp	Efficiency
2.5x10 ⁷	1	0.777269		
2.5x10 ⁷	2	0.356578	2.17980	1.0899
2.5x10 ⁷	3	0.233180	3.33334	1.11111
2.5x10 ⁷	4	0.191311	4.06286	1.01572
2.5x10 ⁷	5	0.163093	4.76580	0.95316
2.5x10 ⁷	6	0.141583	5.48985	0.914975
2.5x10 ⁷	7	0.120029	6.47568	0.925097
2.5x10 ⁷	8	0.108472	7.16562	0.895702

4.1 Efficiency

It is observed from table 4.1 that for $n = 2, 3, 4$ cores, a superlinear efficiency is achieved. Also from cores 4 to 7, the efficiency stabilizes around 0.91 and 0.95 ,($\geq 90\%$) which is considered a good efficiency.

Efficiency remains consistently high, showing that overhead from communication and synchronization is minimal. Therefore, the system scales efficiently.

4.2 SpeedUp Analysis

The parallel program shows excellent scalability up to 8 cores, with some superlinear benefits at low core counts. At lower cores, below four cores, some cores experience superlinear

speedup whereas beyond 4 cores, speedup continues to increase but at a slower rate, showing diminishing returns.

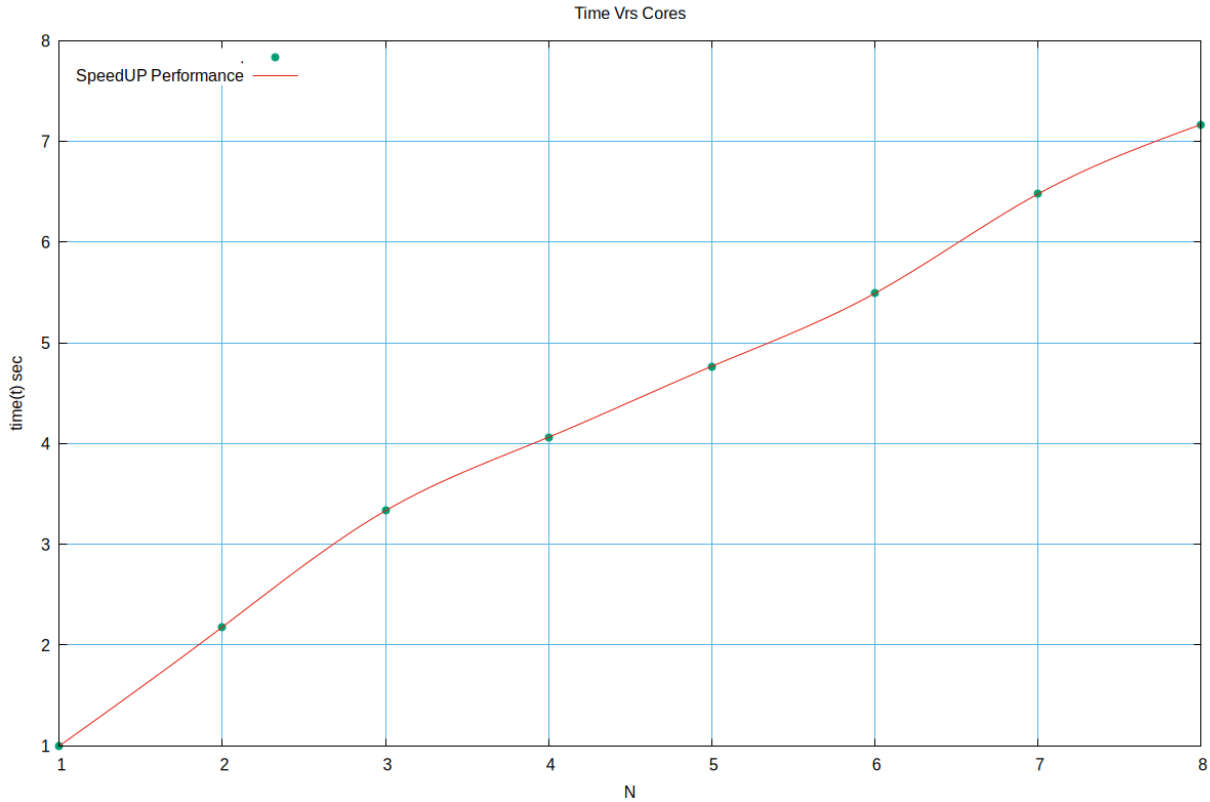


Figure 4.1: Speedup Performance of increasing cores against serial run

From figure 4.1, we observe that the MPI-based parallel implementation of vector-vector multiplication achieves a significant performance improvement as the number of cores increases. The execution time decreases, and the speedup demonstrates near-linear scaling up to the tested core counts. However, diminishing returns may occur as core counts increase further due to communication overhead.

Chapter 5

conclusion

The performance analysis of the MPI-based vector–vector multiplication demonstrates that the parallel implementation achieves strong scalability up to 8 cores. The results show instances of superlinear speedup for small core counts (2–3 cores), attributed to cache effects, followed by consistently high efficiency (around 90%) as the number of cores increases. Although efficiency slightly declines at higher core counts due to communication overheads, the overall performance remains robust. These results indicate that the program is well-optimized for distributed-memory execution and can effectively utilize multiple cores to handle large-scale computations.

Appendix A

MPI Implementation the Dot product of two Vectors.

```
1      #include "vector.h"
2
3  /**
4   * main - Entry point
5   * Description: Executes and does functional calls
6   * Return: Always 0 on success
7   */
8  double start;
9  double end;
10 double *t_used;
11
12 int main(int argc, char **argv){
13     MPI_Init(&argc, &argv);
14     MPI_Status status;
15     int rank, nprocs;
16     int begin, e, count;
17     int s, en, c;
18     double *recVecA;
19     double *recVecB;
20     double vecans;
21     double global_time = 0;
22     double result;
23
24     int i, j, k;
25     FILE *fd;
26     char *filename = "IndexNumber_scale.txt";
27     int m,n, size;
```

APPENDIX A. MPI IMPLEMENTATION THE DOT PRODUCT OF TWO VECTORS.11

```

28     double *vecA;
29     double *vecB;
30     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
31     MPI_Comm_size(MPI_COMM_WORLD, &nprocs);
32
33     if (rank == 0){
34
35         if (nprocs == 1){
36             fd = fopen(filename, "w");
37             fprintf(fd, "%-*s%-*s%-*s\n", SPACE, "N",
38                 SPACE, "Core",SPACE, "V.V/s");
39
40         }
41         else {
42             fd = fopen(filename, "a");
43         }
44         if (fd == NULL){
45             fprintf(stderr, "Unable to open %s", filename
46                 );
47             return (1);
48         }
49         //fprintf(fd, "%-*s %-*s %-*s\n", SPACE, "N", SPACE,
50             "Core",SPACE, "V.V/s");
51
52     }
53     if (argc != 2){
54         fprintf(stderr, "Usage <%s> arg\n", argv[0]);
55         exit(-1);
56     }
57
58     m = atoi(argv[1]);
59     n = m;
60     size = m * n;
61
62     begin = (rank * n) / nprocs;
63     e = ((rank + 1)*n) / nprocs;
64     count = e - begin;
65     recVecA = (double *)malloc(sizeof(double) * count);
66     recVecB = (double *)malloc(sizeof(double) * count);
67     if (rank == 0){
68         vecA = (double *)malloc(sizeof(double) * n);
69         vecB = (double *)malloc(sizeof(double) * n);
70
71         if (vecA != NULL && vecB != NULL){

```

APPENDIX A. MPI IMPLEMENTATION THE DOT PRODUCT OF TWO VECTORS.12

```

68         for (j = 0; j < n; j++){
69             vecA[j] = rand() /((double)RAND_MAX;
70             vecB[j] = rand() /((double)RAND_MAX;
71
72         }
73     }
74     else{
75         fprintf(stderr, "Failed to allocate memory
76             for vectors A and B");
77         exit(-1);
78     }
79     for (i = 1; i < nprocs; i++){
80         s = (i * n) / nprocs;
81         en = ((i + 1) * n) / nprocs;
82         c = en - s;
83         MPI_Send(vecA + s, c, MPI_DOUBLE, i, 0,
84             MPI_COMM_WORLD);
85         MPI_Send(vecB + s, c, MPI_DOUBLE, i, 1,
86             MPI_COMM_WORLD);
87     }
88     for (j = 0; j < count ; j++){
89         recVecA[j] = vecA[begin + j];
90         recVecB[j] = vecB[begin + j];
91     }
92     free(vecA);
93     free(vecB);
94 }
95 else{
96     MPI_Recv(recVecA, count, MPI_DOUBLE, 0, 0,
97         MPI_COMM_WORLD, &status);
98     MPI_Recv(recVecB, count, MPI_DOUBLE, 0, 1,
99         MPI_COMM_WORLD, &status);
100 }
101 MPI_Barrier(MPI_COMM_WORLD);
102
103 vecans = vector_vector(count, count, recVecA, recVecB);
104
105 global_time = end - start;
106
107 MPI_Reduce(&vecans, &result, 1, MPI_DOUBLE, MPI_SUM, 0,
108     MPI_COMM_WORLD);
109 if (rank == 0){

```

APPENDIX A. MPI IMPLEMENTATION THE DOT PRODUCT OF TWO VECTORS.13

```
105         fprintf(fd, "%-*d", SPACE, m);
106         fprintf(fd, "%-*d□%-*f\n",SPACE, nprocs, SPACE,
            global_time);
107
108
109     }
110
111     global_time = 0;
112     free(t_used);
113
114     free(recVecA);
115     free(recVecB);
116
117     if (rank == 0){
118         fclose(fd);
119     }
120     MPI_Finalize();
121     return (0);
122 }
```